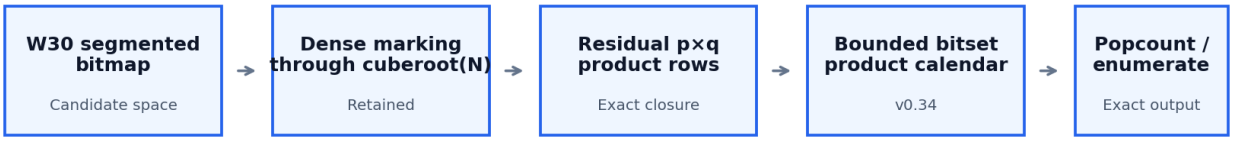
RED2

v0.34

PRODUCT CALENDAR

Public Technical & Benchmark Dossier

Exact full sieve. A bounded event calendar for sparse residual product rows — designed to reduce empty row visits while preserving the complete prime bitmap.



Calendar changes WHEN sparse product rows are revisited — not WHICH composites are marked.

26 / 26	7.73%	24.81%	10^13
local paired wins	same-binary calendar gain	10T row-visit reduction	validated traversal

Prepared from RED2_v0.34_PRODUCT_CALENDAR release evidence

10 September 2026 • Public / GitHub-ready • Source package supplied by Mikhail Zerafa

CLAIM STATUS

Strong measured results. Global “fastest in the world up to 1 trillion” status is NOT established by the current evidence.

01 / EXECUTIVE SUMMARY

What RED2 v0.34 establishes

A high-performance exact full sieve with reproducible correctness evidence and bounded scheduling for sparse product rows.

346,065,536,839	103	157	6.21%
π(10^13), correct	independent 10T sample windows	bitmap validation cases	local 1T gain vs unchanged RED2

RED2 v0.34 preserves the full-sieve contract: every composite represented in the W30 bitmap is explicitly cleared before counting or callback delivery. It does not use a scalar prime-count correction or a probabilistic primality shortcut.

The v0.34 change is a bounded product-row calendar. Busy residual product rows retain direct scanning; sparse rows can be scheduled into future segments using a compact ring bitset. The calendar changes when a row is revisited, not which composite products are removed.

Strongest publishable performance statement In the supplied local matrix, v0.34 beat the unchanged RED2 baseline in all 26 measured pairs across 13 configurations from 10^9 through 10^12. When scheduling was active, paired reductions ranged from 16.49% to 28.96%. A same-binary ablation at 10^11 / 64 KiB / 8 threads isolated a 7.73% scheduling benefit.

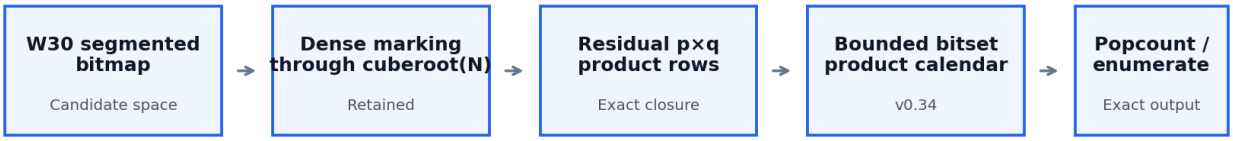
External benchmark status A prior v0.33 target experiment beat the installed primesieve executable at 10^11 on the reported i7-8700K in 20/20 measured pairs across two sessions. The v0.34 archive does not contain a matched v0.34-vs-primesieve run at 10^12. Therefore a global or 10^12 external “fastest” claim is premature.

This dossier deliberately separates three questions: (1) is the bitmap correct? (2) does the v0.34 calendar improve RED2? (3) is RED2 globally fastest? The evidence strongly supports the first two within the measured scope; the third remains open.

02 / ARCHITECTURE

The bounded product-row calendar

Retained mathematics; new scheduling geometry.



Calendar changes WHEN sparse product rows are revisited — not WHICH composites are marked.

Let y = floor(cuberoot(N)). After marking primes through y, any remaining composite is p×q with y < p ≤ q and p ≤ floor(sqrt(N)). RED2 explicitly marks those residual products in the W30 bitmap.

The prior engine revisited every active residual row in every later segment after its p^2 birth. At 512 KiB this produced 3,342,769,123 row visits at 10^12 and 97,407,590,242 at 10^13. Profiling showed large fractions of these visits could be empty.

Direct vs scheduled For segment size B and schedule factor F, rows p ≤ F×B use the retained direct loop. Larger rows use a calendar bitset. F defaults to 2. The threshold is geometric and configurable — it is not a CPU-name special case.

- One bit per scheduled row per ring slot: a set bit means that row has a product due in the represented future segment.
- Due rows are processed in increasing row order; the cursor advances to the first product beyond the current segment, then the row is rescheduled exactly once.
- Ring horizon is derived from sqrt(N), the maximum observed auxiliary prime gap and segment geometry.
- Calendar storage is bounded to 4 MiB per active worker; if the bound cannot be met, the engine falls back to direct scanning.
- Power-of-two segment sizes use shifts; arbitrary admitted byte sizes use exact division. Tests include 5000-byte segments.

Coverage invariant Task territories remain disjoint; row activation is unique; reinsertion always targets a strictly later segment; no event is scheduled beyond its territory. Boundary bits above N retain the existing endpoint handling.

03 / CORRECTNESS

A fast wrong sieve is a failed sieve

v0.34 was checked at bitmap, traversal, sanitizer, fallback and enumeration levels.

Check	Scope	Status
Full 10T count	346,065,536,839	PASS
10T segment callbacks	635,783, each exactly once	PASS
Independent 10T sample windows	103 windows / 431,499,952 represented positions	PASS
Bitmap validation matrix	157 cases	PASS
Portable build	157 cases without -march=native	PASS
Forced calendar-memory fallback	157 cases; eight fallback cases exercised	PASS
ASan / UBSan	1B traversal; leak detection disabled	PASS
Reduced runtime team	7 requested / 2 actual with oracle	PASS
Ordered enumeration	$\pi(10^6)$=78,498; identical ordering	PASS

The 10^13 run is especially important because it traverses the full segment space with invariant checks enabled. The independent oracle comparison is sampled rather than a byte-for-byte second full bitmap, so the evidence should be described exactly that way.

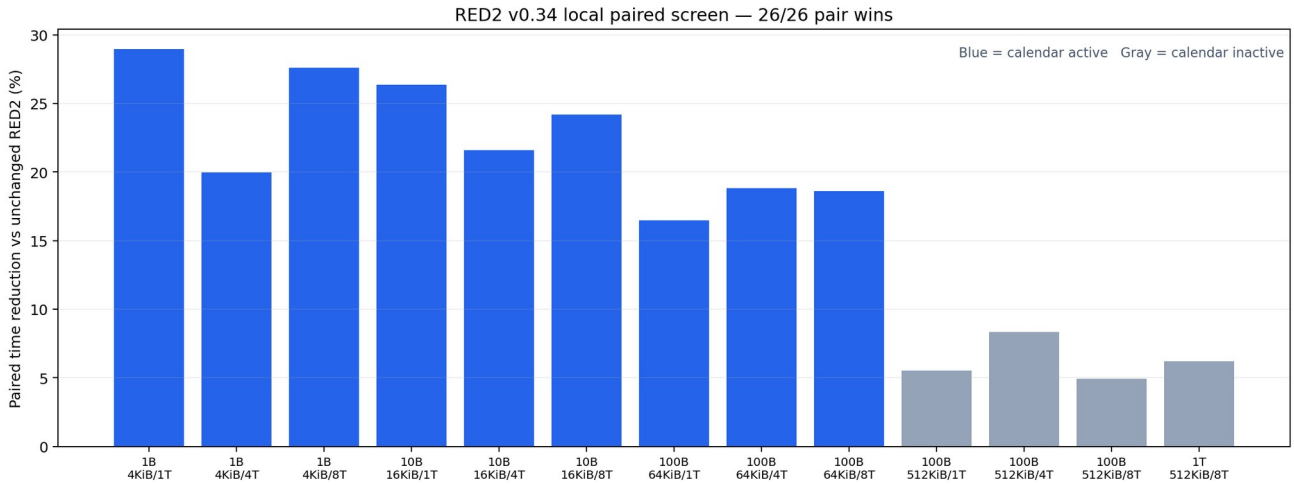
No scalar correction The release README states that every residual composite is explicitly cleared before the count is taken. The independent product audit H is metadata only and is not subtracted from a partial count.

Source and evidence integrity are captured in SHA256SUMS.txt. The release source hash for red2_calendar.c is ffadb44f037465931abc28cec4b393e87580ce5d315883df684560b7602fce80.

04 / PERFORMANCE

Local paired screen

Same compiler flags, sequential configurations, all runs retained.



Workload	Calendar	Paired reduction vs original	Wins
10^9 / 4 KiB	calendar active	19.98–28.96%	6 / 6
10^10 / 16 KiB	calendar active	21.61–26.34%	6 / 6
10^11 / 64 KiB	calendar active	16.49–18.80%	6 / 6
10^11 / 512 KiB	calendar inactive	4.91–8.35%	6 / 6
10^12 / 512 KiB	calendar inactive	6.21%	2 / 2

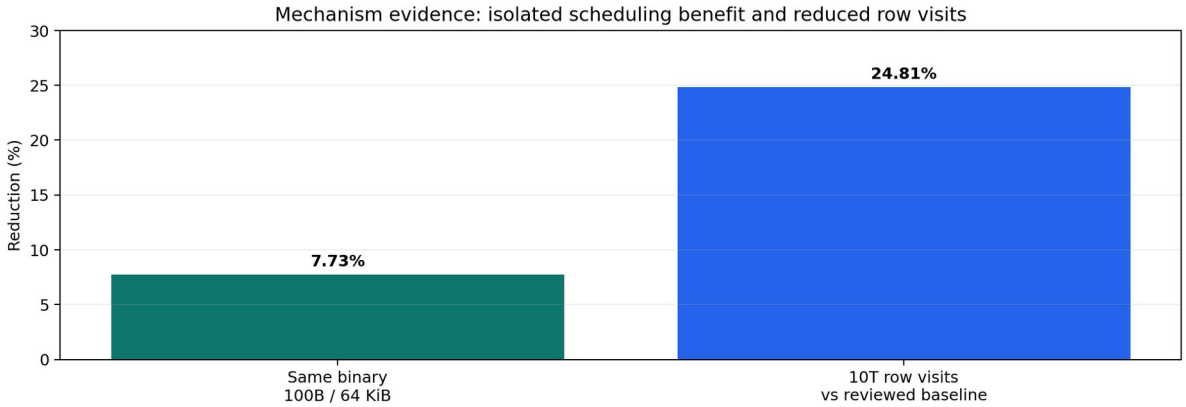
Environment: Linux, GCC 13.3.0, -O3 -march=native, C11/OpenMP. The machine reported AMD EPYC 9V74 while the cgroup limited execution to an eight-CPU budget. These are not results from Mik's i7-8700K and should not be transferred across architectures.

Important 1T nuance At 10^12 with 512 KiB segments, the v0.34 calendar is inactive because the scheduling threshold exceeds sqrt(N). The measured 6.21% gain over unchanged RED2 therefore cannot be attributed to the calendar mechanism itself; layout/code-generation/run-condition effects can contribute.

05 / MECHANISM EVIDENCE

Does the calendar itself help?

A same-binary ablation and a 10T row-visit screen isolate the intended effect better than version-to-version timing alone.



7.728%	2 / 2	24.81%	1.165 MiB
same-binary runtime reduction	ablation pair wins	10T row visits avoided	calendar / active worker at 10T

At N=10^11, 64 KiB segments and eight threads, factor 2 (calendar enabled) was compared with factor 0 (calendar disabled) inside the same candidate binary. Scheduling reduced paired runtime by 7.728466% and won both pairs.

At 10^13, the candidate completed in 453.899 seconds and reduced exact product-row visits from the independently reviewed baseline of 97,407,590,242 to 73,237,843,449 — 24,169,746,793 fewer visits, or 24.81%.

Do not overread the 10T timing The subsequent original-control timing did not leave a saved result, so the archive cannot compute a matched v0.34-vs-original 10T speedup. The row-visit reduction is solid mechanism evidence; the 453.899 s value is a candidate timing, not a comparative headline.

06 / EXTERNAL COMPARISON

What can be claimed publicly?

The safest GitHub release is explicit about scope and comparator class.

20 / 20	6.31–6.91%	10^12	NOT EST.
v0.33 target wins vs primesieve	paired reduction at 10^11	v0.34 direct external test pending	global fastest ranking

The prior v0.33 dossier records two target sessions at N=10^11 on the reported Intel Core i7-8700K. RED2 won all 20 measured pairs against the installed primesieve executable, with paired time reductions of 6.9066% and 6.3075%. This is meaningful same-machine evidence, but it is not a universal leaderboard.

primesieve itself is a mature multithreaded segmented sieve of Eratosthenes with wheel factorization and bucket sieving. primecount is a different comparator class: it uses combinatorial prime-counting algorithms and can be dramatically faster when the task is only π(x), because it does not need to construct a full prime bitmap.

Requested “fastest in the world up to 1 trillion” claim Not supported yet. The supplied v0.34 evidence contains no direct v0.34-vs-primesieve comparison at 10^12, and no multi-hardware independent ranking. Publishing that sentence as fact would overstate the evidence.

Strong GitHub wording today “RED2 v0.34 is an experimental exact full prime sieve. It has been validated through 10^13, won 26/26 local paired comparisons against unchanged RED2 across 10^9–10^12, and builds on a v0.33 target result that beat the installed primesieve executable in 20/20 measured pairs at 10^11 on an i7-8700K. Direct v0.34 external benchmarking at 10^12 is pending.”

07 / REPRODUCTION

The decisive 1T benchmark

One command can promote the claim from “pending” to a reproducible same-machine result.

Windows / MSYS2 UCRT64 Run with other heavy workloads stopped. Keep the generated results ZIP, primesieve version, executable hash, environment and every timed row.

```
RED2_LIMIT=1000000000000 RED2_THREADS=12 RED2_PAIRS=10 \
  bash compare_calendar.sh "C:\Program Files\Primesieve\bin\primesieve.exe"
```

The supplied compare_calendar.sh script validates both RED2 builds, checks an independent oracle, records hashes and alternates measured order. RED2_PAIRS accepts even values; using 10 pairs gives a more publishable sample than the two-pair default.

If v0.34 wins reproducibly at N=10^12, the correct promoted statement is still machine-scoped: “RED2 v0.34 beat the installed primesieve build at N=10^12 on the tested i7-8700K configuration.” A global “fastest” statement would require broader comparable hardware, tuned configurations and independent reproduction.

Comparator discipline Do not collapse primesieve and primecount into one leaderboard. primesieve is the relevant full segmented-sieve comparator. primecount answers a different question: fastest exact π(x) counting without requiring a full prime bitmap.

08 / GITHUB RELEASE

Recommended repository front matter

Strong enough to attract attention; precise enough to survive technical scrutiny.

Headline RED2 v0.34 Product Calendar — exact full prime sieve with bounded sparse-row scheduling

RED2 v0.34 is an experimental C/OpenMP full prime sieve using W30 segmented bitmaps, cube-root residual-product closure and a bounded product-row calendar. The calendar schedules sparse residual rows into future segments while retaining direct scanning for busy rows.

Release evidence validates the exact count π(10^13)=346,065,536,839, 157 bitmap cases, independent sampled 10T windows, sanitizer runs, fallback behavior and ordered enumeration. In the supplied local benchmark matrix v0.34 won 26/26 paired comparisons against unchanged RED2. A same-binary ablation measured a 7.73% scheduling gain at 10^11 / 64 KiB / 8 threads.

Historical target evidence for v0.33 reports 20/20 wins over the installed primesieve executable at 10^11 on an Intel Core i7-8700K. v0.34-vs-primesieve at 10^12 remains a pending release gate.

Repository badge language Correctness: validated to 10^13 | Local A/B: 26/26 wins | Calendar ablation: -7.73% time | External 10^12: pending

Current package note: the source headers retain a proprietary/confidential notice and the release README states that no new license is granted. Publishing source on GitHub makes it publicly visible; choose and document the licensing terms you actually intend before presenting it as open source.

APPENDIX / EVIDENCE REGISTER

Files used for this dossier

All release claims are traceable to the supplied archive or explicitly marked historical/public context.

Source	Use
README.md	Release contract, commands, portability and claim limits
DESIGN.md	Calendar geometry, invariant, ring horizon and memory fallback
LOCAL_RESULTS.md	Local matrix, ablation, 10T correctness and limitations
evidence/matrix_summary.json	13-configuration paired timing summary
evidence/ablation_summary.json	Same-binary schedule-factor 2 vs 0 result
evidence/large_10t_review.json	10T candidate timing, row visits and comparison limitations
evidence/independent_validation_review.json	10T correctness and validation summary
evidence/enumeration_check.txt	Ordered enumeration through 1,000,000
SHA256SUMS.txt	Release file integrity
RED2 v0.33 Technical & Benchmark Dossier	Historical target v0.33-vs-primesieve 10^11 evidence
kimwalisch/primesieve README	Comparator definition: segmented Eratosthenes / wheel / bucket
kimwalisch/primecount README	Comparator-class distinction: combinatorial prime counting

Public comparator references

https://github.com/kimwalisch/primesieve

https://github.com/kimwalisch/primecount

Document status Public technical summary derived from the supplied RED2 v0.34 archive. It is not an independent laboratory certification, patent opinion, or global performance ranking.